

4. Écrire à plusieurs endroits

La position est un calcul comme un autre.

1. Ce qu'il faut comprendre

Trois appels, trois endroits. Rien de neuf — sauf ceci : la colonne et la ligne **ne sont pas obligées d'être écrites en clair**.

`texte(4 + 2, 8, "...")` marche, et `texte(x, y, "...")` avec des variables aussi. C'est ce qui rendra possible tout ce qui bouge.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <texte>` → [0.0.2. La fonction `texte\(\)` — écrire un mot](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **504 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	49 o	← la console, à l'allumage – la mise en route de la console
main()	40 o	← démarrage – ta fonction
VBlank	8 o	← la console, à chaque image – compte les images, 60 fois par seconde
AttendreImage	28 o	← image() – attend l'image suivante
AttendreVBlank	16 o	← démarrage – attend que l'écran soit libre
AttendreAcces	12 o	← EcrireTexte – attend de pouvoir écrire une case
EcrireTexte	10 o	← texte() – écrit un texte, lettre par lettre
CopierTuiles	67 o	← démarrage – copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage – vide les deux cartes de l'écran
Tuiles	224 o	← CopierTuiles – 14 lettres sur 43
"EN HAUT A GAUCHE"	16 o	← texte() – un texte gravé
"AU MILIEU"	9 o	← texte() – un texte gravé
"TOUT EN BAS"	11 o	← texte() – un texte gravé

Lettres gravées : ABCEGHILMNOSTU

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

texte() × 3	33 o sur place, dans main()	+ EcrireTexte (10 o)
image() × 1	3 o sur place, dans main()	+ AttendreImage (28 o)

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// CE PROGRAMME : écrit trois textes à trois endroits de l'écran.
//
// texte(colonne, ligne, "MOT") écrit un mot à l'écran.
//  colonne : de 0 (tout à gauche) à 19 (tout à droite)
//  ligne   : de 0 (tout en haut) à 17 (tout en bas)
// Les lettres permises : A à Z (majuscules, sans accents), 0 à 9, l'espace et ! ? . - : # |

#include <texte> // écrit un texte à l'écran
```

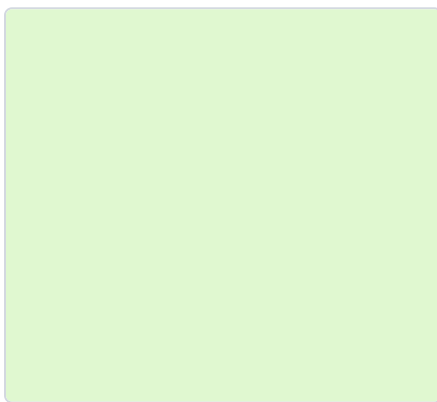
2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {                                // Le jeu commence ici.
    texte(0, 0, "EN HAUT A GAUCHE");        // colonne 0, ligne 0 : le coin en haut à gauche
    texte(4, 8, "AU MILIEU");              // colonne 4, ligne 8 : à peu près au milieu
    texte(2, 17, "TOUT EN BAS");           // ligne 17 : la dernière ligne

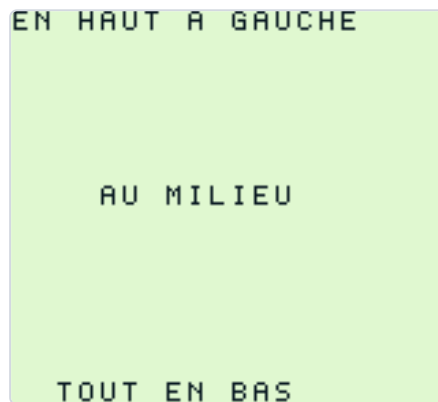
    while (true) {                          // Répète pour toujours :
        image();                            // attend l'image suivante (60 par seconde).
    }                                       // Sans cette boucle, le programme s'arrêterait.
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de commencer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Trois lignes, en haut, au milieu et tout en bas.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ la première touche le coin
- ✓ la deuxième est au milieu
- ✓ la troisième est sur la dernière ligne

Ensuite : **5. Une constante plutôt qu'un nombre** — chapitre 1 : Les tout premiers pas. · [gameboy3](#) — écrire une cartouche Game Boy en C++